

TP1 - BCC264 2026-2

Fork

1.1 Análise do Código

A função `fork()` duplica o processo atual dentro do SO. O processo que chamou a função é o processo pai e o novo processo criado é chamado de processo filho. A função pode retornar tanto o processo pai quanto o processo filho. Sabemos qual é qual devido ao retorno da função.

Com o retorno da função podemos realizar diferentes operações, a depender de qual processo foi retornado pelo fork. Nas linhas 15 e 19 temos verificações do PID.

- Se o pid é 0, então estamos no processo filho e vamos realizar instruções específicas do processo filho.
- Se o pid é maior que 0, estamos no processo pai e realizamos intruções específicas do processo pai.
- Caso o pid seja menor que 0, ocorreu um erro ao criar o novo processo.

Quando executarmos o programa, ambas as saídas de pai e filho devem aparecer no terminal.

```
None
Eu sou o processo pai. Meu PID é %%%
O PID do meu processo filho é %%%
Eu sou o processo filho. Meu PID é %%%
O PID do meu processo pai é %%%
```

1.2 Executando o Código

Percebemos que a saída foi exatamente a mesma, alterando somente os valores dos processos.

```
None
Eu sou o processo pai. Meu PID é 63883
O PID do meu processo filho é 63884
Eu sou o processo filho. Meu PID é 63884
O PID do meu processo pai é 63883
```

Isso acontece porque o programa roda em dois processos diferentes, um processo pai e um processo filho. Então ambas as condicionais são satisfeitas, porém em diferentes processos.

1.3 Sleep

Sem o `sleep()`, os dois processos disputam a CPU para ver qual roda primeiro, então eles não tem uma ordem definida de execução. Já com o `sleep()`, o processo pai tem que esperar 2 segundos (tempo definido por mim) para executar, o que dá tempo do processo filho executar e imprimir suas mensagens primeiro que o processo pai.

1.4 Novo Fork

Serão criados 3 processos no total:

- O pai original (avó)
- O filho
- O filho do filho (neto)

None

```
Eu sou o processo vó. Meu PID é 70953
O PID do meu processo filho é 70954
Eu sou o processo pai. Meu PID é 70954
O PID do meu processo filho é 70955
O PID do meu processo pai é 70953
Eu sou o processo filho. Meu PID é 70955
O PID do meu processo pai é 70954
```

1.5 Condicional Alternada

Os valores impressos pelo pai e pelo filho são iguais, mas a ordem pode ser imprevisível. Devido ao fork, temos duas variáveis `i` separadas, então se o processo pai modificar a variável, ela não afeta a variável do processo filho, isso ocorre devido ao isolamento da memória entre processos.

1.6 Perguntas

1.6.1

A diferença reside no valor retornado pela função `fork`, que retorna o pid do novo filho ou 0 (caso seja o processo pai). Outra diferença é que o processo pai mantém seu próprio pid original. O processo filho recebe um novo PID e também um PPID, que se refere ao PID do seu *parent* (pai) -> *Parent Process ID*.

1.6.2

Eles são únicos porque servem para identificar exclusivamente cada processo no sistema operacional. Não pode haver 2 ou mais processos com o mesmo PID para não causar ambiguidade na hora de consulta ou gerenciamento de processos pelo SO.

1.6.3

Não, os PIDs provavelmente não serão os mesmos. Isso ocorre porque o sistema operacional define de forma sequencial e dinâmica os IDs de cada processo através de uma tabela de IDs disponíveis.

Shell

2.1 Comandos

Executando `ls` -> lista todos os arquivos e pastas do diretório atual

- `fork_bin fork.c shell shell.c`

Executando `pwd` -> obtém o caminho absoluto até o diretório atual

- `/home/phenriqX/Documents/college/classes/bcc264/tp1`

Executando `whoami` -> imprime o usuário logado atualmente

- `phenriqX`

2.2 Diferenças da Questão Anterior

A diferença desse programa para o da questão anterior é que, ao criar um novo fork, usamos o processo filho criado para executar funções digitadas pelo usuário usando a função `execvp()`, que carrega e executa um programa externo ao programa que está rodando. O processo pai usa `waitpid()` para esperar o processo filho concluir antes de pedir o próximo comando.

2.3 Background/Foreground

Modo Foreground -> modo de execução padrão do Shell. O usuário não consegue digitar novos comandos no prompt até que o processo atual finalize.

Modo Background -> É ativado ao adicionar o caractere `&` no final do comando. O Shell cria o processo filho para executar a tarefa, mas não chama `waitpid()`. O Shell imprime o novo prompt, permitindo que o usuário continue trabalhando enquanto outra tarefa é executada.

No programa, colocamos o modo background se passarmos `'&'` ao final da linha. Como o Shell não chama `waitpid()`, o controle do Shell retorna imediatamente para que o usuário possa digitar outros comandos.

C/C++

```
int main() {
    char line[MAX_LINE];           // Linha de comando
    char *args[MAX_ARGS];         // Lista de argumentos
    pid_t pid;
    int status;
    int bg; // flag pro controle de background/foreground

    while (1) {                   // Loop principal do shell
        printf("shell> ");        // Prompt do shell
        if (fgets(line, sizeof(line), stdin) == NULL) { // Lê a linha de comando
            break;                // Sai do loop se fgets falhar
        }

        if (line[0] == '\n') {
            continue;            // Ignora linhas vazias
        }

        line[strcspn(line, "\n")] = '\0'; // Remove o caractere de nova linha

        bg = 0;
        char *bg_ptr = strstr(line, "&");
        if (bg_ptr != NULL) {
            bg = 1;
            *bg_ptr = '\0';
        }
        parse_command(line, args); // Analisa a linha de comando em argumentos

        if (strcmp(args[0], "exit") == 0) {
            break;                // Sai do shell se o comando for "exit"
        }

        pid = fork();             // Cria um novo processo
        if (pid < 0) {
            fprintf(stderr, "Fork falhou\n");
            continue;
        } else if (pid == 0) {
            // Processo filho
            if (execvp(args[0], args) < 0) { // Executa o comando
                fprintf(stderr, "Erro ao executar o comando\n");
                exit(1);
            }
        } else {
            if (bg == 0)
                waitpid(pid, &status, 0);
            else
                // Processo pai
                printf("Processo rodando em background com PID: %d\n", pid);
        }
    }
}
```

```
    }

    return 0;
}
```

Pipes

3.1 Múltiplas Strings

```
C/C++
int main() {
    int pipe1[2], pipe2[2];
    pid_t pid;
    char buffer[BUFFER_SIZE];

    // Cria os pipes
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid = fork(); // Cria um novo processo
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) { // Processo filho
        close(pipe1[1]); // Fecha o lado de escrita do primeiro pipe
        close(pipe2[0]); // Fecha o lado de leitura do segundo pipe

        while (1) {
            ssize_t num_bytes = read(pipe1[0], buffer, BUFFER_SIZE); // Lê a
string do pai
            if (num_bytes <= 0)
                break;
            if (strcmp(buffer, "exit") == 0)
                break;

            to_uppercase(buffer); // Converte para maiúsculas
        }
    }
}
```

```

        write(pipe2[1], buffer, strlen(buffer) + 1); // Envia a string
convertida para o pai
    }

    close(pipe1[0]);
    close(pipe2[1]);
} else { // Processo pai
    close(pipe1[0]); // Fecha o lado de leitura do primeiro pipe
    close(pipe2[1]); // Fecha o lado de escrita do segundo pipe

    while (1) {
        printf("Digite uma string: ");
        fflush(stdout);

        fgets(buffer, BUFFER_SIZE, stdin);

        buffer[strcspn(buffer, "\n")] = '\0'; // Remove o caractere de nova
linha

        write(pipe1[1], buffer, strlen(buffer) + 1); // Envia a string para o
filho

        if (strcmp(buffer, "exit") == 0)
            break;

        read(pipe2[0], buffer, BUFFER_SIZE); // Lê a string convertida do
filho
        printf("String convertida: %s\n", buffer); // Exibe a string
convertida
    }
    close(pipe1[1]);
    close(pipe2[0]);
}
return 0;
}

```

É adicionado um loop infinito `while (1)` tanto no processo pai quanto no filho. Quando o usuário digita "exit", o processo pai escreve essa instrução no pipe1 e interrompe seu loop. Quando o filho lê a mensagem, reconhece o comando e também encerra seu ciclo.

3.2 Terceiro Processo

```

C/C++
int main() {
    int pipe1[2], pipe2[2], pipe3[2];

```

```

pid_t pid, pid2;
char buffer[BUFFER_SIZE];

// Cria os pipes
if (pipe(pipe1) == -1 || pipe(pipe2) == -1 || pipe(pipe3) == -1) {
    perror("pipe");
    exit(EXIT_FAILURE);
}

pid = fork(); // Cria um novo processo
if (pid < 0) {
    perror("fork");
    exit(EXIT_FAILURE);
}

if (pid == 0) { // Processo filho
    close(pipe1[1]); // Fecha o lado de escrita do primeiro pipe
    close(pipe2[0]); // Fecha o lado de leitura do segundo pipe
    close(pipe3[0]); // Não utiliza o pipe3
    close(pipe3[1]);

    while (1) {
        ssize_t num_bytes = read(pipe1[0], buffer, BUFFER_SIZE); // Lê a
string do pai
        if (num_bytes <= 0)
            break;
        if (strcmp(buffer, "exit") == 0) {
            write(pipe2[1], buffer, strlen(buffer) + 1);
            break;
        }

        to_uppercase(buffer); // Converte para maiúsculas
        write(pipe2[1], buffer, strlen(buffer) + 1); // Envia a string
convertida para o pai
    }

    close(pipe1[0]);
    close(pipe2[1]);
} else { // Processo pai
    pid2 = fork();

    if (pid2 < 0) {
        perror("fork1");
        exit(EXIT_FAILURE);
    }

    if (pid2 == 0) {
        close(pipe1[0]); // Não utiliza o pipe1
        close(pipe1[1]);
    }
}

```

```

close(pipe2[1]); // Não escreve no pipe2
close(pipe3[0]); // Não lê do pipe3

while (1) {
    ssize_t bytes_read = read(pipe2[0], buffer, BUFFER_SIZE);
    if (bytes_read <= 0) break;

    if (strcmp(buffer, "exit") == 0) {
        break;
    }

    // 2ª Transformação: Inversão (Ex: UFOP -> POFU)
    reverse_string(buffer);

    // Envia a string invertida de volta ao Pai via pipe3
    write(pipe3[1], buffer, strlen(buffer) + 1);
}

close(pipe2[0]);
close(pipe3[1]);
exit(EXIT_SUCCESS);
} else {
    // Fecha as extremidades não utilizadas pelo Pai
    close(pipe1[0]); // Não lê do pipe1
    close(pipe2[0]); // Não utiliza o pipe2
    close(pipe2[1]);
    close(pipe3[1]); // Não escreve no pipe3

    while (1) {
        printf("Digite uma string (ou 'exit' para sair): ");
        fflush(stdout);

        if (fgets(buffer, BUFFER_SIZE, stdin) == NULL) {
            break;
        }

        buffer[strcspn(buffer, "\n")] = '\0';

        // Envia a string original para o Filho 1 via pipe1
        write(pipe1[1], buffer, strlen(buffer) + 1);

        if (strcmp(buffer, "exit") == 0) {
            break;
        }

        read(pipe3[0], buffer, BUFFER_SIZE);
        printf("String final (Maiúsculas + Invertida): %s\n\n", buffer);
    }
}

```

```

    }

    close(pipe1[1]);
    close(pipe3[0]);
}
}
return 0;
}

```

Com mais um processo, é necessário maior manipulação dos pipes de comunicação, sendo preciso fechar as portas que não seriam usadas, isso evita deadlocks. O processo pai lê a string, envia para o filho 1 através do pipe1, esse filho 1 converte para maiúsculo e escreve o resultado no pipe2 e repassa para o filho 2. O filho 2 lê a string do pipe2 e reverte a string e envia a string invertida para o processo pai via pipe3.

Threads

4.1 Threads e Comportamento do Código

Uma thread é a menor unidade de execução que um sistema operacional pode gerenciar dentro de um processo "pesado". Múltiplas threads dentro do mesmo processo compartilham a mesma memória, ou seja, não há isolamento de memória.

Nó código, o valor final de `contador_global` é sempre diferente porque as duas threads criadas pelo programa tentam acessar e modificar a variável ao mesmo tempo, o que cria um race condition (condição de corrida). Uma thread lê o valor da variável antes que a outra tenha a atualizado, o que faz com que um dos incrementos seja perdido.

4.2 Mutex

O uso de Mutex nos ajuda a evitar as race conditions. Quando a primeira thread chega em `pthread_mutex_lock()`, ela dá um lock (fecha) no mutex e entra na seção crítica para alterar a variável. Se a outra thread tentar ao mesmo tempo fazer o lock do mutex (`pthread_mutex_lock`) enquanto a outra ainda está em lock, ela é colocada em modo de espera. Assim que a outra thread acaba de incrementar a variável, ela executa o `pthread_mutex_unlock()`, que destranca o mutex e permite que a outra thread faça o lock para poder modificar a variável com segurança. Com isso, a instrução de incremento torna-se isolada e protegida, garantindo um resultado final correto em todas as execuções.

Link do Código no GDBOnline

<https://onlinegdb.com/uIXGG9TbH>

Link do Vídeo:

<https://youtu.be/vO-bzKFG4X4>